

CS1CA Computer Systems Architecture

Information Representation-5

Week 2 lecture 2 of the autumn semester 2025

Lectured by

Dr. Hong Wei

h.wei@reading.ac.uk

Room 133 in Polly Vacher Building



Outline of the topic

- Introduction: what is information?
- Decimal, binary and hexadecimal number systems
 - Conversion between number systems
- Internal representation of characters
- Internal representation of integers
- Arithmetic and logic operations
- **Internal representation of real numbers**
 - IEEE 754 floating point format

In this lecture

- The following contents are delivered.
 - Internal representation of real numbers
 - IEEE 754 floating point format
- The learning outcomes are listed below.
 - To be able to use the IEEE 754 floating point format to represent real decimal numbers in bit patterns
 - To be aware of rounding errors in the format

Internal representation of real numbers

- A real number such as 25.62_{10} can be written in exponential form, 2.562×10^1 .

- An arbitrary real number, A , can be written as

$$A = m \times R^e$$

m : the mantissa; e : the exponent; R : the base.

- The mantissa contains the digits operated on directly.
- The exponent gives the magnitude.
- In computer science, it is known as **floating point**.

IEEE 754 floating point format – 1

- This is a standard implemented in the hardware of almost all modern processors;
- Real number A represented by

$$A = m \times 2^e$$

where $m = 0$ when $A = 0$,

$$m = 1.<fraction> \text{ when } A \neq 0$$

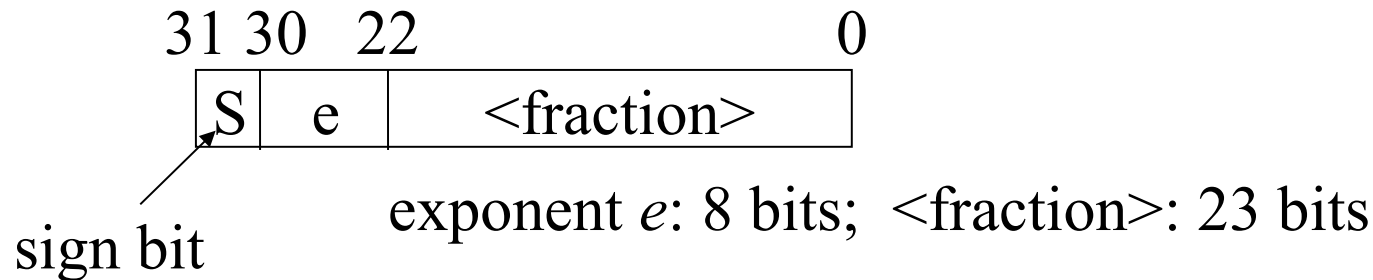
Example:

$$13.02 = 1.6275 \times 2^3$$

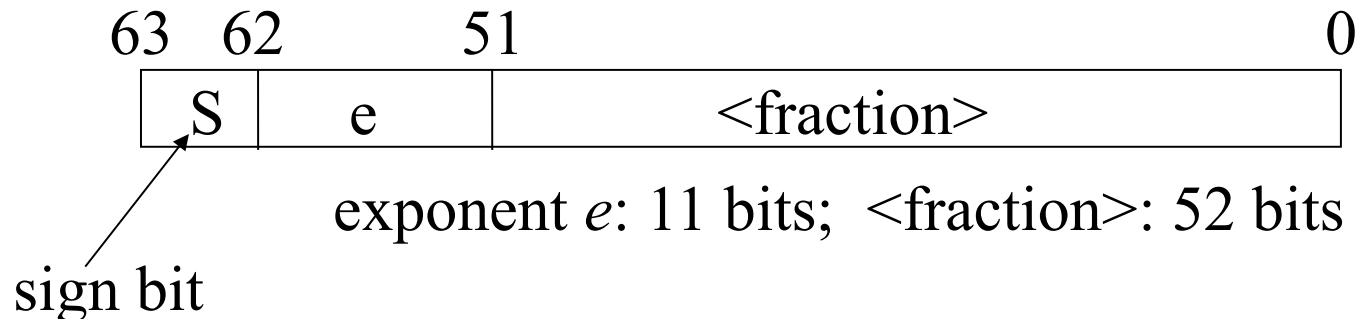
where $m = 1.6275$, $fraction = 0.6275$, and $e = 3$.

IEEE 754 floating point format – 2

- Two formats of the representation
 - Single precision floating point (32 bits)



- Double precision floating point (64 bits)



IEEE 754 floating point format – 3

- The sign bit S is set as 1 when A is negative, otherwise reset as 0;
- From the previous example, we know that to evaluate e , the original real number must be normalised.
- If $A = i . f$ (i : integer, f : fraction), then the IEEE exponent e must satisfy the equation:

$$i \text{ div } 2^e = 1$$

Example: $A = 25.62$, to make $25 \text{ div } 2^e = 1$, we need to find out e . Apply “ $25 \div 2$ ” iteratively until the integer becomes to 1. The number of iterations is the exponent $e = 4$.

- The exponent e can be positive ($i > 1$), negative ($i < 1$) or zero ($i = 1$).

More examples of real numbers

- Write real numbers in the way with which mantissa and exponents are used, and the base is 2.
- **A real number: -6.22**
 - $i = 6$, it should satisfy $6 \text{ div } 2^e = 1$
 - Do $6 \div 2$ until the integer part becomes to 1. The number of iteration determines $e = 2$; mantissa = $6.22 \div 2^e = 1.555$
 - Finally, $-6.22 = -1.555 \times 2^2$
- **A real number: 0.07**
 - To normalise 0.07, it should satisfy $0.07 \text{ div } 2^e = 1$
 - **e must be a negative number.** It becomes 0.07×2^{-e}
 - Do 0.07×2 iteratively until the integral part of the result becomes to 1. The number of the iterations determines e .
 - In this case, there are 4 iterations so that $0.07 \times 2^4 = 1.12$ ($e = -4$).
 - Finally, $0.07 = 1.12 \times 2^{-4}$

How to deal with a negative exponent?

- Exponent e has a fixed length of bits, *e.g.* 8 bits for a single floating point.
- IEEE 754 chose to represent the exponent e as $e + \text{offset}$, where $\text{offset} = 2^{n-1} - 1$ and n is the number of bits allocated to e .
 - For SFP, $n = 8$ then the $\text{offset} = 127$.
 - For DFP, $n = 11$ then the $\text{offset} = 1023$
- In this way, a negative e is dealt with. Let's see some examples.

Floating point exponent expression *examples*

- For single precision, e field: 8-bits,
offset = $127_{10} = 01111111_2$
 $e = 3 \quad 3 + 127 = 130 = 10000010_2$
 $e = -6 \quad -6 + 127 = 121 = 01111001_2$
- For double precision, e field: 11-bits
offset = $1023_{10} = 01111111111_2$
 $e = 3 \quad 3 + 1023 = 1026 = 100000000010_2$
 $e = -6 \quad -6 + 1023 = 1017 = 011111111001_2$

IEEE floating point: converting the mantissa to binary

- Mantissa is equal to

$$m = A \div 2^e = 1. \langle fraction \rangle$$

Example: $25.62 \div 2^4 = 1.60125$

$\langle fraction \rangle = 0.60125$

- $\langle fraction \rangle$ has a fixed length of bits, e.g. 23 bits for a single floating point.
- The task became as converting the $\langle fraction \rangle$ part into binary.
- There is an algorithm to transform external decimal fraction to internal binary format.

Conversion of decimal fraction to any base R representation

- We have the following expression,

$$\left\langle 0.f_1f_2\ldots \right\rangle_{10} = r_1 \times R^{-1} + r_2 R^{-2} + r_3 R^{-3} + \ldots$$

- Consider multiplying both sides by R

$$\left\langle 0.f_1f_2\ldots \right\rangle_{10} \times R = r_1 + r_2 R^{-1} + r_3 R^{-2} + \ldots$$

- Repeating the procedure yields coefficients r_i

Conversion Algorithm: from decimal fraction to binary representation

$F_0 = \text{original } \langle \text{fraction} \rangle;$

$j := 1;$

REPEAT;

$I_j := \text{integer part of } (R \times F_{j-1});$

$F_j := \text{fractional part of } (R \times F_{j-1});$

record I_j ;

$j := j + 1;$

UNTIL $j = \text{number of digits required};$

Example 1 of the $\langle fraction \rangle$ conversion algorithm: decimal to **binary representation**

- Convert 0.1875 to an $R = 2$ representation.

$$F_0 = 0.1875$$

j	$R \times F_{j-1}$	I_j	F_j
1	$2 \times 0.1875 = 0.375$	0	
	0.375		
2	$2 \times 0.375 = 0.75$	0	0.75
3	$2 \times 0.75 = 1.5$	1	0.5
4	$2 \times 0.5 = 1.0$	1	0.0

$$\langle 0.1875 \rangle_{10} = \langle 0.0011 \rangle_2$$

Example 2 of conversion algorithm: decimal to **hex representation**

- Convert $F_0 = 0.60125$ to an $R = 16$ representation.

j	$R \times F_{j-1}$	I_j	F_j
1	9.62	9	0.62
2	9.92	9	0.92
3	14.72	E	0.72
4	11.52	B	0.52
5	8.32	8	0.32
6	5.12	5	0.12
7	1.92	1	0.92
8		...	...

24-bit boundary

$\langle 0.60125 \rangle_{10} = \langle 0.99EB851... \rangle_{16}$

Rounding of approximate binary forms

- Most conversions will not be exact. The representation must fit into 23 bits (single reals);
- We can either truncate after 23 bits or round using the 24th bit;
- Rounding introduces less error than truncating.

24 bit hex.		9	9	E	B	8	5
24 bit bin.	1001	1001	1110	1011	1000	0101	
Rounding to 23 bits		100	1100	1111	0101	1100	
0011							

The procedure of a rounding mode

- To round a 24 bit binary to a 23 bit binary, the following stages are followed.
 - Right-shift the binary string for 1 bit (lost the least significant bit), and leave the 24th bit empty.
 - Examine the lost least significant bit
 - If the lost bit is 1, add 1 to the current least significant bit as a 23 bit binary, and carry forward if needed.
 - If the lost bit is 0, keep the 23 bits as it is.

Rounding error

- If a rounding occurs, a small error is introduced.
- Rounding from 24 bits to 23 bits introduces an error which is bounded by 2^{-24} .
- 2^{-24} is 0.000000006, which means single reals are accurate to 7 decimal digits;
- After operations (+, -, $\times$...) this error increases.

Put it all together

- The three components of the number

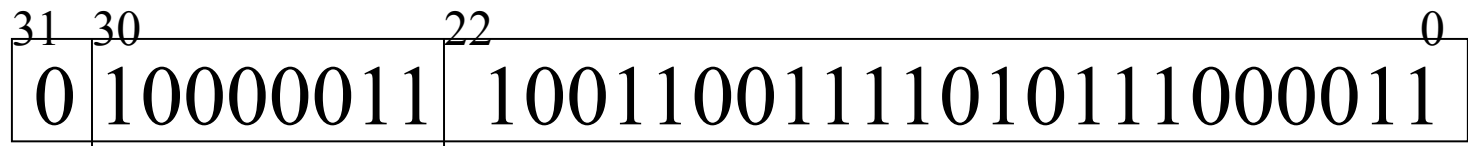
$$25.62 = 1.60125 \times 2^4$$

can now be assembled as $S = 0$

$$\text{exponent} = 4 + 127 = 131_{10} = 10000011_2$$

$$\text{fraction} = 0.60125_{10} = 10011001111010111000011_2$$

- We have 25.62 in binary as



End of the lecture

Self-assessment Exercises

- Convert the following decimal numbers to IEEE floating point format (single precision: 32 bits):

(a) 10.025 (b) -186.32 (c) 0.0875

End of the topic

- Summarise what you have learned under the topic of Information Representation.
- Complete the self-assessed exercises (formative) after each lecture.
- Make sure you have knowledge to do the exercises.
- Read the recommended books, and explore more resources to support your learning, such as the Internet.